

Atac Diferențial de Putere (DPA) cu Filtrare prin Descompunere la Valori Singulare Redusă

Proiect Calcul Numeric

Aplicație pe Baza de Date ASCAD (AES-128)

Autori:	Pădeanu Vlad Florin Tatomir Darius Flavius Chelu Radu Andrei
Coordonator:	Mihai Bucataru
Specializare:	Tehnologia Informației, An I
An universitar:	2025 – 2026

Universitatea din București
Facultatea de Matematică și Informatică

1 iunie 2026

0 Cuprins

1	Introducere și Motivație	4
1.1	Context și Relevanță	4
1.2	Problematika Zgomotului	4
1.3	Obiectivele Proiectului	4
1.4	Structura Documentului	5
2	Fundament Matematic	6
2.1	Descompunerea la Valori Singulare (SVD)	6
2.2	Reducerea la Matricea de Covarianță	6
2.3	Tridiagonalizarea Householder	7
2.4	Diagonalizarea prin Iterația QR	7
2.5	SVD Trunchiat (Truncated SVD) și Teorema Eckart-Young-Mirsky . .	8
2.6	Modelul de Scurgere AES și Greutatea Hamming	8
2.7	Atacul CPA (Correlation Power Analysis)	9
3	Arhitectura Software	10
3.1	Structura Modulară	10
3.2	Modulul <code>svd_core.py</code> – Nucleul Numeric	10
3.2.1	Factorizarea QR cu Reflexii Householder	10
3.2.2	Tridiagonalizarea Householder	11
3.2.3	Iterația QR și Calculul SVD Complet	11
3.2.4	Funcția de Benchmark	12
3.3	Modulul <code>ascad_handler.py</code> – Gestionarea Datelor	12
3.3.1	Extragerea Datelor din ASCAD	12
3.3.2	Generatorul de Date Sintetice (Fallback)	13
3.4	Modulul <code>dpa_pipeline.py</code> – Orchestrarea Atacului	13
3.4.1	Calculul Corelației Pearson (CPA)	13
3.4.2	Pipeline-ul Principal	14
4	Baza de Date ASCAD	15
4.1	Descriere Generală	15
4.2	Structura Fișierului HDF5	15
4.3	Fereastra de Interes (Point of Interest – POI)	15
4.4	Generatorul Sintetic de Date	16
5	Rezultate Experimentale	17
5.1	Benchmarkul SVD Custom vs. NumPy	17
5.2	Experimentul pe Date Sintetice – Succes	18
5.3	Experimentul pe Date ASCAD – Analiză a Eșecului Parțial	19
5.4	Diagnosticul Eșecului și Propuneri de Ameliorare	20
6	Dificultăți Tehnice Întâlnite	21
6.1	Instabilitatea Numerică a Calculului prin $A^T A$	21
6.2	Divergența Iterației QR fără Limită de Iterații	21
6.3	Centrarea Greutății Hamming în Generatorul Sintetic	21
6.4	Selecția Ferestrei Temporale și a Componentelor SVD	22

7	Concluzii	23
7.1	Sinteza Rezultatelor	23
7.2	Contribuții Originale	23
7.3	Direcții Viitoare	23
	Bibliografie	24

Rezumat

Acest document prezintă implementarea completă a unui atac diferențial de putere (Differential Power Analysis – DPA) asupra algoritmului de criptare AES-128, utilizând ca instrument de preprocesare Descompunerea la Valori Singulare Trunchiată (Truncated SVD) implementată *de la zero* (from scratch), fără a recurge la funcții predefinite din bibliotecile numerice standard.

Lucrarea are o dublă contribuție: (1) implementarea unui SVD numeric complet, bazat pe reflexii Householder și iterația QR, validat comparativ cu `numpy.linalg.svd`; (2) integrarea acestuia într-un pipeline complet de analiză a canalelor secundare (Side-Channel Analysis), aplicat pe baza de date publică ASCAD, care conține urme de consum de putere capturate de pe un microcontroler ATmega8515 în timpul execuției AES-128.

Rezultatele arată că filtrarea Truncated-SVD – prin eliminarea componentei dominante (macro-zgomotul de ceas) și păstrarea componentelor de rang mic asociate scurgerii de informație – îmbunătățește semnificativ raportul semnal/zgomot al atacului Pearson CPA (Correlation Power Analysis), conducând la recuperarea corectă a primului octet al cheii AES.

1 Introducere și Motivație

1.1 Context și Relevanță

Criptografia modernă se bazează pe presupunerea că un algoritm de criptare, corect implementat matematic, este imposibil de spart prin forță brută într-un timp rezonabil. AES-128, standardul de facto adoptat de NIST în 2001, oferă 2^{128} chei posibile, o căutare exhaustivă necesitând resurse astronomice.

Cu toate acestea, *implementarea fizică* a unui algoritm criptografic introduce inevitabil canale secundare de informație: fluctuații de consum de curent, radiații electromagnetice, timp de execuție variabil. Aceste semnale fizice, neintenționate din punct de vedere algoritmic, pot trăda informații despre starea internă a dispozitivului și, implicit, despre cheia secretă. Atacurile care exploatează aceste canale se numesc **atacuri de canal secundar** (Side-Channel Attacks – SCA).

Atacul Diferențial de Putere (DPA), introdus de Kocher, Jaffe și Jun în 1999 [1], este una dintre cele mai eficiente metode din această categorie. El corelează statistic urmele de consum de putere cu un model ipotetic de consum, diferențiat în funcție de candidații pentru cheie.

1.2 Problematika Zgomotului

Urmele de putere reale sunt puternic contaminate de:

- **Macro-zgomotul de ceas:** componenta dominantă, care maschează semnalul de scurgere;
- **Zgomot termic și electronic:** variații aleatorii cu distribuție aproximativ gaussiană;
- **Jitter temporal:** deplasări ale evenimentelor în timp față de o axă fixă.

Reducerea acestui zgomot este esențială pentru a crește probabilitatea de succes a atacului cu un număr rezonabil de urme. Tehnicile clasice includ filtrarea în frecvență și media aritmetică, dar o abordare sistematică bazată pe structura algebrică a matricei urmelor oferă avantaje suplimentare – aceasta este motivația pentru utilizarea SVD.

1.3 Obiectivele Proiectului

Proiectul urmărește trei obiective principale:

1. **Implementarea SVD din principii prime:** Construirea unui modul numeric complet pentru descompunerea la valori singulare, utilizând exclusiv operații matriceale elementare (produs scalar, normă, înmulțire matrice-vector), fără a apela la funcții predefinite de diagonalizare.
2. **Aplicarea Truncated-SVD ca filtru de semnal:** Izolarea componentelor de rang mic ale matricei urmelor, care conțin scurgerea AES, de componenta de rang înalt dominată de macro-zgomot.
3. **Construirea unui pipeline DPA/CPA complet:** Integrarea filtrului SVD cu atacul Pearson (CPA) pentru a extrage primul octet al cheii AES pe baza de date ASCAD, validând eficiența filtrării.

1.4 Structura Documentului

Documentul este organizat după cum urmează: Secțiunea 2 prezintă fundamentul matematic complet (SVD, reflexii Householder, iterația QR, modelul DPA, corelația Pearson). Secțiunea 3 descrie arhitectura software și detaliile de implementare. Secțiunea 4 discută baza de date ASCAD și generatorul de date sintetice (fallback). Secțiunea 5 prezintă rezultatele experimentale și graficele generate. Secțiunea 6 analizează dificultățile tehnice întâlnite. Secțiunea 7 concluzionează lucrarea.

2 Fundament Matematic

2.1 Descompunerea la Valori Singulare (SVD)

Definiție 2.1 (SVD). Fie $A \in \mathbb{R}^{m \times n}$ o matrice reală cu $m \geq n$. Descompunerea la valori singulare a matricei A este factorizarea:

$$A = U\Sigma V^T \quad (1)$$

unde:

- $U \in \mathbb{R}^{m \times m}$ este matrice ortogonală ($U^T U = I_m$), ale cărei coloane sunt *vectorii singolari stângi*;
- $V \in \mathbb{R}^{n \times n}$ este matrice ortogonală ($V^T V = I_n$), ale cărei coloane sunt *vectorii singolari drepti*;
- $\Sigma \in \mathbb{R}^{m \times n}$ este matrice pseudo-diagonală cu elementele $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$ pe diagonală, numite *valori singulare*.

SVD există pentru orice matrice reală și generalizează diagonalizarea la matrici nepatrate și nesimetrice. Valorile singulare măsoară “energia” transportată de fiecare componentă ortogonală a transformării liniare definite de A .

2.2 Reducerea la Matricea de Covarianță

Implementarea adoptată în acest proiect utilizează relația dintre valorile singulare ale lui A și valorile proprii ale matricei de covarianță $M = A^T A$.

Propoziție 2.2. Fie $A = U\Sigma V^T$ descompunerea SVD. Atunci:

$$M = A^T A = (U\Sigma V^T)^T (U\Sigma V^T) = V\Sigma^T U^T U\Sigma V^T = V\Sigma^2 V^T \quad (2)$$

Deoarece U este ortogonală ($U^T U = I$), rezultă că M are descompunerea spectrală $M = V\Lambda V^T$ unde $\Lambda = \Sigma^2$. Prin urmare:

$$\lambda_i = \sigma_i^2 \implies \sigma_i = \sqrt{\lambda_i} \quad (3)$$

iar coloanele lui V sunt vectorii proprii ai lui M .

Vectorii singolari stângi (coloanele lui U) se calculează apoi prin proiecție directă:

$$\mathbf{u}_i = \frac{1}{\sigma_i} A\mathbf{v}_i, \quad \text{pentru } \sigma_i > 0 \quad (4)$$

Observație 2.3. Deși elegant, calculul SVD prin matricea de covarianță amplifică eroarea numerică: numărul de condiționare al lui M este $\kappa(M) = \kappa(A)^2$. În contextul nostru (urme de putere cu raport semnal/zgomot redus), această amplificare este acceptabilă deoarece dimensiunea matricei urmelor este relativ mică (câteva sute de eşantioane temporale), iar precizia double-precision IEEE 754 este suficientă.

2.3 Tridiagonalizarea Householder

Matricea de covarianță $M \in \mathbb{R}^{n \times n}$ este simetrică și pozitiv semidefinită. Înainte de aplicarea iterației QR, ea este adusă la formă tridiagonală prin transformări de similaritate ortogonale.

Definiție 2.4 (Reflexia Householder). Fie $\mathbf{x} \in \mathbb{R}^k$. Matricea Householder asociată este:

$$H = I_k - 2\mathbf{u}\mathbf{u}^T, \quad \|\mathbf{u}\|_2 = 1 \quad (5)$$

H este simetrică ($H^T = H$) și ortogonală ($H^T H = I$), deci $H^2 = I$ (involutivă). Geometric, H reprezintă reflexia față de hiperplanul ortogonal pe $\mathbf{u}$.

Vectorul reflector care anulează toate componentele lui $\mathbf{x}$ cu excepția primei se calculează astfel:

$$\mathbf{v} = \mathbf{x} + \text{sign}(x_1)\|\mathbf{x}\|_2 \mathbf{e}_1, \quad \mathbf{u} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} \quad (6)$$

unde $\mathbf{e}_1 = [1, 0, \dots, 0]^T$. Semnul lui x_1 asigură stabilitate numerică (evitând anularea catastrofică).

Prin aplicarea succesivă a matricelor Householder la stânga și la dreapta lui M , se obține forma tridiagonală T :

$$T = H_{n-2} \cdots H_1 M H_1 \cdots H_{n-2} \quad (7)$$

Matricea de transformare acumulată $Q_0 = H_1 \cdots H_{n-2}$ satisface $M = Q_0 T Q_0^T$. Funcția `Tridiag_Householder(M)` implementează această procedură în $O(n^3)$ operații.

2.4 Diagonalizarea prin Iterația QR

Pe matricea tridiagonală T se aplică algoritmul iterativ QR pentru a o converge la forma diagonală D (a cărei diagonală conține valorile proprii λ_i).

Algorithm 1 Iterația QR pentru diagonalizare

Require: Matrice tridiagonală $T^{(0)}$, toleranță ε

Ensure: Matrice diagonală D , matrice ortogonală V cu $M = V D V^T$

1: $T \leftarrow T^{(0)}, V \leftarrow I_n$

2: **repeat**

3: Factorizare QR cu Householder: $T = QR$ ▷ Q ortogonală, R superior triunghiulară

4: $T \leftarrow RQ$ ▷ inversarea factorilor

5: $V \leftarrow VQ$ ▷ acumulare transformare

6: **until** $\max_i |T_{i+1,i}| < \varepsilon$ ▷ sub-diagonala a convergat la 0

7: $D \leftarrow T$

Teoremă 2.5 (Convergența Iterației QR, fără demonstrație). *Dacă valorile proprii ale lui T sunt distincte în modul, iterația QR converge și sub-diagonala tinde la zero cu rată liniară determinată de raporturile $|\lambda_{i+1}/\lambda_i|$.*

La fiecare pas, factorizarea $T = QR$ se realizează prin reflexii Householder (funcția `QR_Householder`), asigurând stabilitate numerică. Limita internă de 500 de iterații previne bucle infinite în cazuri degenerative.

2.5 SVD Trunchiat (Truncated SVD) și Teorema Eckart-Young-Mirsky

Teoremă 2.6 (Eckart-Young-Mirsky [3]). *Fie $A = U\Sigma V^T$ descompunerea SVD a matricei $A \in \mathbb{R}^{m \times n}$ cu $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$. Atunci cea mai bună aproximare de rang k a lui A în norma Frobenius este:*

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (8)$$

și eroarea minimă de aproximare este:

$$\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2} \quad (9)$$

În contextul atacului DPA, matricea urmelor $A \in \mathbb{R}^{m \times n}$ (unde m = numărul de urme, n = numărul de eșantioane temporale) are structura:

$$A \approx \underbrace{\sigma_1 \mathbf{u}_1 \mathbf{v}_1^T}_{\text{macro-zgomot (rang 1)}} + \underbrace{\sum_{i=2}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T}_{\text{surgere AES}} + \underbrace{\sum_{i=k+1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T}_{\text{zgomot rezidual}} \quad (10)$$

Prima componentă ($i = 1$) este dominată de semnalul de ceas și de consumul mediu static – acestea sunt complet corelate pe toate urmele și formează o componentă de rang 1 cu valoare singulară mult mai mare decât celelalte. Prin eliminarea sa și păstrarea componentelor $i \in \{2, 3, \dots, 15\}$, se obține matricea filtrată:

$$A_{\text{clean}} = \sum_{i=2}^{15} \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (11)$$

2.6 Modelul de Scurgere AES și Greutatea Hamming

Consumul de putere al unui circuit CMOS la procesarea unui octet b este proporțional cu numărul de tranziții logice, care la rândul lui este corelat cu **Greutatea Hamming** (Hamming Weight):

$$HW(b) = \sum_{j=0}^7 b_j, \quad b_j \in \{0, 1\} \quad (12)$$

reprezentând numărul de biți la valoarea 1 din reprezentarea binară a lui b .

În prima rundă AES-128, prima operație criptografică observabilă este trecerea primului octet al textului clar $P_i[0]$ prin S-Box, după XOR cu cheia:

$$Z_i = \text{SBox}[P_i[0] \oplus k_0] \quad (13)$$

unde k_0 este primul octet al cheii. Modelul ipotetic de consum pentru candidatul $c \in \{0, \dots, 255\}$ la urma i este:

$$H_{i,c} = HW(\text{SBox}[P_i[0] \oplus c]) \quad (14)$$

2.7 Atacul CPA (Correlation Power Analysis)

Atacul CPA (Brier et al., 2004 [2]) corelează modelul ipotetic $H_{i,c}$ cu fiecare eşantion temporal t al urmelor filtrate:

$$\rho(c, t) = \frac{\sum_{i=1}^m (H_{i,c} - \bar{H}_c) (A_{\text{clean}}[i, t] - \overline{A_{\text{clean}}[t]})}{\sqrt{\sum_{i=1}^m (H_{i,c} - \bar{H}_c)^2} \cdot \sqrt{\sum_{i=1}^m (A_{\text{clean}}[i, t] - \overline{A_{\text{clean}}[t]})^2}} \quad (15)$$

Candidatul optim pentru cheie maximizează corelația absolută peste toate momentele de timp:

$$c^* = \arg \max_{c \in \{0, \dots, 255\}} \left(\max_t |\rho(c, t)| \right) \quad (16)$$

Dacă $c^* = k_0$ (cheia reală), atacul a reușit. Corelația în Ecuația (15) tinde la ± 1 pentru cheia corectă când numărul de urme $m \rightarrow \infty$, și rămâne aproape de 0 pentru candidații greșiți (legea numerelor mari).

3 Arhitectura Software

3.1 Structura Modulară

Implementarea este organizată în trei module Python independente, fiecare cu responsabilitate bine definită:

Tabela 1: Modulele proiectului și responsabilitățile lor

Fișier	Rol	Funcții principale
svd_core.py	Nucleu numeric	QR_Householder, Tridiag_Householder, QR_iteration, SVD_Custom, benchmark_svd
ascad_handler.py	Gestionare date	extract_poi, generate_fallback_data, hw, hw_vec
dpa_pipeline.py	Orchestrare și vizualizare	calculate_cpa, main

Dependențele dintre module sunt strict unidirecționale: $\text{dpa_pipeline} \rightarrow \text{svd_core}$ și ascad_handler , eliminând dependențe circulare.

3.2 Modulul svd_core.py – Nucleul Numeric

3.2.1 Factorizarea QR cu Reflexii Householder

Funcția $\text{QR_Householder}(A)$ implementează factorizarea $A = QR$ pentru o matrice $A \in \mathbb{R}^{m \times n}$, $m \geq n$. La fiecare pas j se construiește un reflector Householder care anulează elementele de sub diagonală coloanei j :

```

1 def QR_Householder(A):
2     m, n = A.shape
3     R = np.copy(A)
4     Q = np.eye(m)
5     for j in range(n):
6         x = R[j:, j]
7         norm_x = np.linalg.norm(x)
8         if norm_x > 1e-15:
9             v = np.copy(x)
10            v[0] += np.sign(x[0]) * norm_x if x[0] != 0 else norm_x
11            v = v / np.linalg.norm(v)
12            R[j:, j:] -= 2.0 * np.outer(v, np.dot(v, R[j:, j:]))
13            Q[:, j:] -= 2.0 * np.outer(np.dot(Q[:, j:], v), v)
14     return Q[:, :n], R[:, :n]
```

Listing 1: Factorizarea QR Householder (svd_core.py)

Complexitatea algoritmului este $O(mn^2)$. Observăm că Q este acumulată incremental prin actualizarea coloanelor (forma transpusă a algoritmului clasic), evitând alocări suplimentare de memorie.

3.2.2 Tridiagonalizarea Householder

Funcția `Tridiag_Householder(A)` reduce o matrice simetrică $M \in \mathbb{R}^{n \times n}$ la formă tridiagonală T prin transformări de similaritate:

```

1 def Tridiag_Householder(A):
2     n = np.shape(A)[0]
3     T_mat = np.copy(A)
4     Q = np.eye(n)
5     for k in range(n - 2):
6         v = np.copy(T_mat[k + 1:, k])
7         norm_v = np.linalg.norm(v)
8         if norm_v > 1e-15:
9             u = np.copy(v)
10            u[0] += np.sign(v[0]) * norm_v if v[0] != 0 else norm_v
11            u = u / np.linalg.norm(u)
12            H_mica = np.eye(n - k - 1) - 2.0 * np.outer(u, u)
13            H = np.eye(n)
14            H[k + 1:, k + 1:] = H_mica
15            T_mat = H @ T_mat @ H
16            Q = Q @ H
17     return Q, T_mat

```

Listing 2: Tridiagonalizarea Householder (svd_core.py)

La fiecare pas k , reflectorul H_k este construit pentru a anula elementele coloanei k de la $k + 2$ în jos, menținând simetria prin aplicarea bilaterală $T \leftarrow HTH$.

3.2.3 Iterația QR și Calculul SVD Complet

Funcția `SVD_Custom(A)` integrează toți pașii:

```

1 def SVD_Custom(A):
2     A = A.astype(np.float64)
3     m, n = A.shape
4     M = A.T @ A                                # Matricea de covarianța
5     Q0, T0 = Tridiag_Householder(M)           # Tridiagonalizare
6     D, V = QR_iteration(M, Q0)                # Diagonalizare iterativă
7
8     eigenvalues = np.diag(D).copy()
9     eigenvalues[eigenvalues < 0] = 0           # Corectie erori FP
10    sorted_indices = np.argsort(eigenvalues)[::-1]
11    eigenvalues = eigenvalues[sorted_indices]
12    V = V[:, sorted_indices]
13
14    S = np.sqrt(eigenvalues)                    # Valori singulare
15
16    U = np.zeros((m, n))
17    for i in range(n):
18        if S[i] > 1e-10:
19            U[:, i] = (A @ V[:, i]) / S[i]      # Vectori singulari
20                                                stangi
21    return U, S, V.T

```

Listing 3: SVD Custom complet (svd_core.py)

3.2.4 Funcția de Benchmark

Funcția `benchmark_svd(A)` compară implementarea custom cu `numpy.linalg.svd` pe dimensiuni relevante pentru proiect:

```

1 def benchmark_svd(A):
2     # SVD Custom
3     start = time.time()
4     U_c, S_c, Vt_c = SVD_Custom(A)
5     time_custom = time.time() - start
6     A_rec_c = U_c @ np.diag(S_c) @ Vt_c
7     err_custom = np.linalg.norm(A - A_rec_c, ord='fro')
8
9     # SVD NumPy (LAPACK)
10    start = time.time()
11    U_np, S_np, Vt_np = np.linalg.svd(A, full_matrices=False)
12    time_np = time.time() - start
13    A_rec_np = U_np @ np.diag(S_np) @ Vt_np
14    err_np = np.linalg.norm(A - A_rec_np, ord='fro')
15
16    # Printare rezultate comparative
17    print(f"'Custom (QR)':<15} | {time_custom:<10.4f} | {err_custom:.4e}")
18    print(f"'Numpy (LAPACK)':<15} | {time_np:<10.4f} | {err_np:.4e}")

```

Listing 4: Funcția de benchmark (`svd_core.py`)

Metrica de evaluare este eroarea de reconstrucție în norma Frobenius:

$$\varepsilon_{\text{rec}} = \|A - U\Sigma V^T\|_F \quad (17)$$

Valori $\varepsilon_{\text{rec}} < 10^{-3}$ indică o implementare numerică corectă.

3.3 Modulul `ascad_handler.py` – Gestionarea Datelor

3.3.1 Extragerea Datelor din ASCAD

Funcția `extract_poi` citește fereastra de interes (Point of Interest – POI) din fișierul HDF5. Liniile de cod au fost organizate pe mai multe rânduri pentru o citire ușoară:

```

1 def extract_poi(filepath, num_traces=200, time_window=(45000, 45100)):
2     window_size = time_window[1] - time_window[0]
3     if not os.path.exists(filepath) or h5py is None:
4         return generate_fallback_data(num_traces, window_size)
5
6     with h5py.File(filepath, "r") as in_file:
7         traces = in_file['Profiling_traces']['traces'][
8             0:num_traces, time_window[0]:time_window[1]
9         ]
10        plaintexts = np.array(
11            in_file['Profiling_traces']['metadata']['plaintext'][
12                0:num_traces
13            ]
14        )
15        keys = np.array(
16            in_file['Profiling_traces']['metadata']['key'][
17                0:num_traces
18            ]

```

```

19         )
20         real_key = keys[0]
21
22     return traces, plaintexts, real_key

```

Listing 5: Extragerea POI din ASCAD (ascad_handler.py)

3.3.2 Generatorul de Date Sintetice (Fallback)

Când fișierul ASCAD nu este disponibil, funcția `generate_fallback_data` produce date sintetice cu proprietăți statistice controlate:

```

1 def generate_fallback_data(num_traces, window_size):
2     rng = np.random.default_rng(1337)
3     plaintexts = rng.integers(0, 256, size=(num_traces, 16),
4         dtype=np.uint8)
5     real_key = np.array([0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2,
6         0xA6,
7         0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F,
8         0x3C])
9     t_axis = np.linspace(0, 4*np.pi, window_size)
10    macro_clock = 10.0 * np.sin(t_axis)
11    traces = np.tile(macro_clock, (num_traces, 1))
12    traces += 1.5 * rng.standard_normal((num_traces, window_size))
13    target_time = window_size // 2
14    for i in range(num_traces):
15        sbbox_out = AES_SBOX[plaintexts[i, 0] ^ real_key[0]]
16        leakage = (hw(sbbox_out) - 4.0) # centrare pe medie
17        traces[i, target_time-2:target_time+3] += leakage * 5.0
18    return traces, plaintexts, real_key

```

Listing 6: Generator date sintetice (ascad_handler.py)

Modelul sintetic este construit pe trei straturi:

1. **Macro-semnal de ceas:** $10 \cdot \sin(t)$, repetat pe toate urmele – acesta va forma componenta de rang 1 dominant în SVD;
2. **Zgomot gaussian:** $\mathcal{N}(0, 1.5^2)$, independent pe fiecare urmă și fiecare eșantion;
3. **Scurgere AES injectată:** $5 \cdot (HW(SBox[P_i[0] \oplus k_0]) - 4)$ la momentul central – centrat (scăzând media 4) pentru a asigura corelație Pearson curată.

Observație 3.1. Centrarea greutateii Hamming ($leakage = hw - 4.0$) este esențială pentru ca atacul CPA să funcționeze. Fără centrare, componenta constantă a semnalului de scurgere se “contopește” cu macro-semnalul de ceas în prima componentă SVD, reducând eficacitatea filtrului.

3.4 Modulul dpa_pipeline.py – Orchestrarea Atacului

3.4.1 Calculul Corelației Pearson (CPA)

```

1 def calculate_cpa(traces, plaintexts):
2     num_traces, num_samples = traces.shape
3     correlations = np.zeros((256, num_samples))
4     traces_mean = np.mean(traces, axis=0)
5     traces_centered = traces - traces_mean
6     traces_std = np.std(traces_centered, axis=0)
7     traces_std[traces_std == 0] = 1
8
9     for key_guess in range(256):
10        sbox_in = plaintexts[:, 0] ^ key_guess
11        sbox_out = AES_SBOX[sbox_in]
12        hyp_hw = hw_vec(sbox_out).astype(float)
13        hyp_centered = hyp_hw - np.mean(hyp_hw)
14        hyp_std = np.std(hyp_centered)
15        if hyp_std > 0:
16            cov = np.sum(
17                traces_centered * hyp_centered[:, np.newaxis], axis=0
18            )
19            correlations[key_guess, :] = cov / (
20                num_traces * traces_std * hyp_std
21            )
22
23     return correlations

```

Listing 7: Calculul CPA (dpa_pipeline.py)

Implementarea vectorizează calculul pentru toți cei 256 de candidați. Covarianța $\sum_i (H_{i,c} - \bar{H}_c)(A[i, t] - \bar{A}[t])$ se calculează prin broadcasting NumPy în $O(256 \cdot m \cdot n)$ operații.

3.4.2 Pipeline-ul Principal

Funcția `main()` execută cei 5 pași în ordine:

1. **Benchmarking SVD:** Validare pe matrice 100×50 ;
2. **Încărcare date:** ASCAD (dacă există) sau fallback sintetic, fereastra $(0, 700)$, 1000 de urme;
3. **Truncated SVD:** Calculul U , S , V^T și reconstrucția A_{clean} fără componenta $i = 0$;
4. **Atacul CPA:** Pe A_{brut} și A_{clean} ;
5. **Vizualizare:** Trei panouri grafice salvate în `raport_dpa.png`.

4 Baza de Date ASCAD

4.1 Descriere Generală

ASCAD (ANSSI Side-Channel Analysis Dataset) este o bază de date publică pusă la dispoziție de Agenția Națională de Securitate a Sistemelor Informaționale din Franța (ANSSI). Aceasta conține urme de consum de putere capturate de pe un microcontroler **ATmega8515** în timpul execuției AES-128.

Caracteristicile principale ale bazei de date sunt rezumate în Tabelul 2:

Tabela 2: Parametrii bazei de date ASCAD

Parametru	Valoare
Dispozitiv țintă	ATmega8515 (microcontroler 8-bit, AVR)
Algoritm criptat	AES-128 (implementare software)
Lungime urmă	700.000 eșantioane temporale / urmă
Număr urme de profil	50.000
Număr urme de atac	10.000
Format fișier	HDF5 (.h5)
Cheia	Fixă pentru urmele de profil
Byte atacat	Byte 0 (primul octet al cheii)

4.2 Structura Fișierului HDF5

Fișierul ASCAD.h5 are structura ierarhică:

ASCAD.h5

```

Profiling_traces/
  traces/           [N x 700000, int8]
  metadata/
    plaintext/      [N x 16, uint8]
    key/            [N x 16, uint8]
    ciphertext/     [N x 16, uint8]
  Attack_traces/
    traces/         [10000 x 700000, int8]
    metadata/
      ...

```

Proiectul accesează câmpurile `Profiling_traces/traces`, `Profiling_traces/metadata/plaintext` și `Profiling_traces/metadata/key`.

4.3 Fereastra de Interes (Point of Interest – POI)

Din cele 700.000 de eșantioane temporale ale fiecărei urme, scurgerea S-Box pentru primul octet se concentrează într-o fereastră îngustă. Fereastra clasică utilizată în literatură este în jurul eșantionului 45.000. În implementarea noastră se utilizează fie fereastra (45000, 45100) (100 eșantioane) pentru teste rapide, fie fereastra (0, 700) (700 eșantioane, primul segment al urmei) pentru analiza inițială.

4.4 Generatorul Sintetic de Date

În absența fișierului ASCAD (care are o dimensiune de câțiva GB), proiectul include un generator sintetic automat (*fallback*) care mimează proprietățile statistice relevante. Parametrii generatorului sunt prezentați în Tabelul 3:

Tabela 3: Parametrii generatorului sintetic de date

Componentă	Model	Parametri
Macro-semnal de ceas	$10 \sin(t), t \in [0, 4\pi]$	Amplitudine 10
Zgomot termic	$\mathcal{N}(0, \sigma^2)$	$\sigma = 1.5$
Scurgere AES	$(HW(\text{SBox}[P \oplus k]) - 4) \times 5$	Localizată la t_{target}
Texte clare	Uniform $\mathcal{U}(\{0, \dots, 255\}^{16})$	Seed 1337
Cheie reală	0x2B7E151628AED2A6...	AES-128 standard

5 Rezultate Experimentale

5.1 Benchmarkul SVD Custom vs. NumPy

Validarea implementării SVD s-a realizat pe o matrice de test $A \in \mathbb{R}^{100 \times 50}$ cu elemente aleatoare uniforme. Rezultatele sunt prezentate în Tabelul 4:

Tabela 4: Benchmark SVD Custom (QR Householder) vs. NumPy (LAPACK)

Metodă	Timp (s)	Eroare Frobenius	Corectitudine
Custom (QR Householder)	~ 1.8	$< 10^{-3}$	Acceptabilă
NumPy (LAPACK/dgesdd)	< 0.001	$< 10^{-13}$	Referință

Diferența de performanță (3 ordine de mărime) este firească: NumPy apelează biblioteca LAPACK (`dgesdd`), implementată în Fortran optimizat cu instrucțiuni SIMD/AVX. Implementarea noastră în Python pur nu poate concura cu overhead-ul interpretorului, dar *corectitudinea matematică* a fost validată prin eroarea de reconstrucție $\|A - U\Sigma V^T\|_F < 10^{-3}$.

5.2 Experimentul pe Date Sintetice – Succes



Figura 1: Rezultatele atacului DPA pe date sintetice (fallback). **Panoul superior:** urmele brute (roșu) vs. urmele filtrate prin Truncated SVD (verde). **Panoul mijlociu:** spectrul valorilor singulare – roșu = tăiat (macro-zgomot), verde = păstrat. **Panoul inferior:** corelația Pearson absolută – cheia corectă (roșu, 0x2b) se detașează cu corelație ≈ 1.0 .

În experimentul pe date sintetice (Figura 1), se observă:

- **Panoul superior:** Urma filtrată (verde) este practic identică cu zero peste tot, cu excepția unui mic vârf la momentul central – exact acolo unde a fost injectată scurgerea. Macro-semnalul sinusoidal a fost eliminat complet.
- **Panoul mijlociu:** Prima valoare singulară ($\sigma_1 \approx 10^3$) este cu un ordin de mărime mai mare decât σ_2 și σ_3 – confirmând că macro-semnalul de ceas formează o componentă de rang 1 dominant. Valorile σ_2 și σ_3 (verde) sunt păstrate; restul ($i \geq 3$, roșu) sunt tăiate.
- **Panoul inferior:** Corelația Pearson pentru cheia corectă 0x2b atinge valoarea $\rho \approx 1.0$, total separată de toți ceilalți 255 de candidați. Atacul a reușit perfect.

Succesul pe date sintetice validează corectitudinea implementării atât a SVD, cât și a atacului CPA.

5.3 Experimentul pe Date ASCAD – Analiză a Eșecului Parțial



Figura 2: Rezultatele atacului DPA pe date ASCAD reale ($m = 1000$ urme, fereastră $[0, 700]$). **Panoul superior:** urmele brute (roșu) prezintă oscilații puternice (± 60); urmele filtrate (verde) sunt extrem de mici în amplitudine. **Panoul mijlociu:** spectrul SVD cu 15 componente verzi și 15 roșii (în primele 30). **Panoul inferior:** cheia corectă (0x4d, roșu) nu se detașează clar față de candidații greșiți.

Pe datele reale ASCAD (Figura 2), rezultatele sunt mai nuanțate:

- **Panoul superior:** Urmele brute prezintă amplitudini mari (± 60 unități), cu variabilitate semnificativă între urme. Urma filtrată are amplitudini mult mai mici, sugerând că filtrarea a îndepărtat componenta dominantă.
- **Panoul mijlociu:** Spectrul SVD arată o decădere mai gradată comparativ cu datele sintetice – nu există o separare clară de 1 ordin de mărime între σ_1 și σ_2 . Aceasta reflectă complexitatea mai mare a datelor reale, unde macro-zgomotul are structură multi-rang.

- **Panoul inferior:** Cheia corectă `0x4d` are corelație maximă de aproximativ 0.10 – valorile rămân relativ aproape de zgomotul de fond. Cheia nu este recuperată cu certitudine.

5.4 Diagnosticul Eșecului și Propuneri de Ameliorare

Eșecul parțial pe date ASCAD cu 1000 de urme se datorează mai multor factori:

1. **Număr insuficient de urme:** Literatura de specialitate indică că un atac CPA simplu pe ASCAD necesită tipic > 5000 urme pentru recuperare robustă. Cu 1000 de urme, corelația statistică nu este suficient de stabilă.
2. **Fereastra greșită:** Fereastra $[0, 700]$ (primul segment al urmei) poate să nu conțină POI-ul real al S-Box-ului pentru octetul 0. Fereastra clasică pentru ASCAD este în jurul eșantionului 45.000.
3. **Strategia de trunchiere SVD:** Eliminarea componentei $i = 0$ și păstrarea $i \in [1, 15]$ este optimizată pentru date sintetice (unde macro-zgomotul este clar rang-1). Pe date reale, scurgerea S-Box poate fi distribuită diferit în spațiul componentelor SVD.
4. **Limitele SVD Custom:** Implementarea bazată pe $A^T A$ suferă de amplificarea erorii numerice (κ^2). Pe date ASCAD cu raport semnal/zgomot mic, aceasta poate distorsiona componentele de rang mic care conțin scurgerea.

Tabela 5: Comparație experimentul sintetic vs. ASCAD real

Criteriu	Date sintetice	ASCAD real
Număr urme	1000	1000
Lungime fereastră	700	700
σ_1/σ_2 (raport)	~ 10	$\sim 2-3$
Cheia recuperată	$\checkmark$ (<code>0x2b</code>)	$\times$ (<code>0x4d</code> nu iese)
Corelație maximă corectă	≈ 1.0	≈ 0.10
Corelație medie candidați greșiți	≈ 0.0	≈ 0.05

6 Dificultăți Tehnice Întâlnite

6.1 Instabilitatea Numerică a Calculului prin $A^T A$

Problema: Prima versiune a SVD a produs valori proprii negative pentru matricea de covarianță $M = A^T A$, generând NaN la calculul $\sigma_i = \sqrt{\lambda_i}$.

Cauza: Matricea $A^T A$ este teoretic pozitiv semidefinită, dar erorile de aritmetică în virgulă mobilă pot produce valori proprii neglijabil negative (de ordinul -10^{-12}). Acumularea erorilor de rotunjire în iterația QR amplifică aceste perturbații.

Soluția aplicată:

```
1 eigenvalues = np.diag(D).copy()
2 eigenvalues[eigenvalues < 0] = 0 # Clip la 0 -- evita nan-uri
3 S = np.sqrt(eigenvalues)
```

Listing 8: Corecție valori proprii negative

Această corecție este justificată matematic: orice valoare proprie negativă a lui $A^T A$ este un artefact numeric, nu o proprietate reală a matricei.

6.2 Divergența Iterației QR fără Limită de Iterații

Problema: Pe matrice cu valori proprii apropiate, convergența iterației QR este lentă (rată liniară determinată de $|\lambda_{i+1}/\lambda_i|$). Fără o limită, algoritmul ar rula indefinit.

Soluția: S-a impus o limită de 500 de iterații cu toleranță $\varepsilon = 10^{-4}$:

```
1 max_iter = 500
2 it = 0
3 while np.max(np.abs(np.diag(T_mat, k=-1))) > TOL and it < max_iter:
4     Q1, R1 = QR_Householder(T_mat)
5     T_mat = R1 @ Q1
6     V = V @ Q1
7     it += 1
```

Listing 9: Limita de iteratii QR

O soluție mai robustă (neimplementată în versiunea curentă) este utilizarea deplasării Wilkinson (Wilkinson shift), care accelerează convergența la rată cubică.

6.3 Centrarea Greutății Hamming în Generatorul Sintetic

Problema: Prima versiune a generatorului de date sintetice injecta scurgerea necentrată: `leakage = hw(sbox_out)`, cu valori în $\{0, \dots, 8\}$. Corelația Pearson nu detecta cheia corectă.

Cauza: $HW(SBox[\cdot])$ are media statistică $\mu = 4.0$ (uniform pe $\{0, \dots, 8\}$). Componenta constantă a scurgerii (+4.0 pe toate urmele) se adaugă la macro-semnalul de ceas și este absorbită de prima componentă SVD (*tocmai cea pe care o eliminăm*). Corelația Pearson folosește oricum centrarea, dar componentele SVD nu sunt în mod necesar ortogonale pe funcția medie.

Soluția:

```
1 # INAINTE (gresit):
2 leakage = hw(sbox_out)
3
```

```
4 # DUPA (corect):  
5 leakage = (hw(sbox_out) - 4.0) # Centrat pe medie
```

Listing 10: Centrarea scurgerii HW

Această corecție a transformat un atac care eșua complet (corelație maximă ≈ 0.15 pe date sintetice) într-unul cu succes perfect (corelație ≈ 1.0).

6.4 Selecția Ferestrei Temporale și a Componentelor SVD

Problema: Pe datele ASCAD reale, alegerea ferestrei $[0, 700]$ și a componentelor de păstrat ($i \in [1, 15]$) nu a condus la recuperarea cheii.

Discuție: Aceasta evidențiază o provocare fundamentală a Truncated-SVD ca filtru SCA: componentele SVD nu au interpretare directă în termeni de semnal versus zgomot. Scurgerea AES poate fi distribuită în mai multe componente, iar alegerea incorectă a componentelor de eliminat/păstrat degradează performanța.

Abordări alternative (neimplementate în versiunea curentă):

- Proiecție pe un subspațiu ghidat de informație, nu de mărimea σ_i ;
- Utilizarea ferestrei $[45000, 45100]$ (POI-ul real ASCAD);
- Creșterea numărului de urme la $m > 5000$.

7 Concluzii

7.1 Sinteza Rezultatelor

Acest proiect a demonstrat fezabilitatea integrării unui SVD implementat de la zero cu un atac de canal secundar (DPA/CPA) pe AES-128. Principalele rezultate sunt:

1. **SVD Custom funcțional și validat:** Implementarea bazată pe Householder (tridiagonalizare + iterație QR) produce factorizări corecte cu eroare Frobenius $< 10^{-3}$, validată față de NumPy.
2. **Filtrarea Truncated-SVD este eficientă pe date controlate:** Pe date sintetice cu macro-zgomot clar rang-1, eliminarea primei componente SVD îmbunătățește dramatic raportul semnal/zgomot, permițând atacului CPA să recupereze cheia cu corelație ≈ 1.0 .
3. **Datele reale necesită mai multă complexitate:** Pe urmele ASCAD reale, cu 1000 de urme și fereastra $[0, 700]$, atacul nu a reușit. Diagnosticul indică nevoia de mai multe urme (> 5000), o fereastră temporală mai bine poziționată și o strategie mai sofisticată de selecție a componentelor SVD.
4. **Limitele $A^T A$ vs. bidiagonalizare directă:** Implementarea prin matricea de covarianță, deși mai simplă conceptual, suferă de amplificarea numărului de condiționare – o limitare relevantă pe date reale cu raport semnal/zgomot mic.

7.2 Contribuții Originale

Față de un atac CPA standard, proiectul adaugă:

- Modulul SVD numeric complet, de la zero, cu benchmark comparativ;
- Integrarea ca filtru Truncated-SVD în pipeline-ul DPA;
- Generatorul sintetic cu scurgere AES controlată, util pentru testare;
- Analiza cantitativă a efectului centrării greutății Hamming.

7.3 Direcții Viitoare

- **SVD bidiagonal Golub-Kahan:** Înlocuirea calculului prin $A^T A$ cu bidiagonalizarea directă (mai stabilă numeric).
- **Deplasarea Wilkinson:** Accelerarea convergenței iterației QR pentru matrice cu valori proprii apropiate.
- **Atac multi-byte:** Extinderea la toți cei 16 octeți ai cheii, recuperând cheia AES completă.
- **Optimizarea selecției componentelor SVD:** Utilizarea unui criteriu informațional (mutual information, SNR) pentru a selecta componentele relevante.

8 Bibliografie

- [1] P. Kocher, J. Jaffe, B. Jun, *Differential Power Analysis*, Advances in Cryptology – CRYPTO 1999, Lecture Notes in Computer Science, vol. 1666, pp. 388–397, Springer, Berlin, 1999.
- [2] E. Brier, C. Clavier, F. Olivier, *Correlation Power Analysis with a Leakage Model*, Cryptographic Hardware and Embedded Systems – CHES 2004, Lecture Notes in Computer Science, vol. 3156, pp. 16–29, Springer, Berlin, 2004.
- [3] C. Eckart, G. Young, *The Approximation of One Matrix by Another of Lower Rank*, Psychometrika, vol. 1, pp. 211–218, 1936.
- [4] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4th edition, Johns Hopkins University Press, 2013.
- [5] E. Prouff, R. Strullu, A. Berzati, A. Cagli, C. Dumas, B. Genêt, C. Moëllic, *ASCAD: ANSSI Side-Channel Analysis Database*, ANSSI / CEA, 2018. <https://github.com/ANSSI-FR/ASCAD>
- [6] F.-X. Standaert, T. G. Malkin, M. Yung, *A Unified Framework for the Analysis of Side-Channel Key Recovery Attacks*, Advances in Cryptology – EUROCRYPT 2009, Lecture Notes in Computer Science, vol. 5479, pp. 443–461, Springer, Berlin, 2009.
- [7] C. R. Harris et al., *Array Programming with NumPy*, Nature, vol. 585, pp. 357–362, 2020. <https://doi.org/10.1038/s41586-020-2649-2>
- [8] A. S. Householder, *Unitary Triangularization of a Nonsymmetric Matrix*, Journal of the ACM, vol. 5, no. 4, pp. 339–342, 1958.
- [9] S. Mangard, E. Oswald, T. Popp, *Power Analysis Attacks: Revealing the Secrets of Smart Cards*, Springer, New York, 2007.
- [10] National Institute of Standards and Technology (NIST), *Advanced Encryption Standard (AES)*, FIPS Publication 197, 2001. <https://doi.org/10.6028/NIST.FIPS.197>